

Aprender de los Logs — Análisis completo de la Sesión 20 (Módulo 7)

Fuente base: "Aprender de los Logs — Observabilidad, memoria de experiencia y prompts que se reescriben" (`learning_from_logs.pdf` , 44 diapositivas) — Módulo 7 (Aprendizaje y Mejora), Programa en Diseño e Implementación de Agentes IA, UTEC Posgrado. Dictada por **Dr. Vicente Machaca Arceda**, el mismo docente de la Sesión 19 (`Sesion19_FeedbackYCorreccion_ANALISIS_COMPLETO.md`) — mismo estilo de seminario académico densamente citado, construido sobre un *survey* de investigación como columna vertebral.

Nota técnica 1: el PDF está protegido con contraseña de propietario (bloquea copiar/editar), lo cual impidió leerlo con el extractor de PDF estándar. Se extrajo el texto completo con `pdftotext` (Poppler), que sí pudo abrirlo porque la restricción es de permisos, no de apertura — las 44 páginas tienen capa de texto completa y limpia, sin necesidad de interpretar imágenes.

Nota técnica 2: el material referencia dos veces el archivo `notebooks/11_logs.ipynb` (diapositivas 38 y 39, "Agente 1: instrumentado" y "Agente 2: mini-ACE") como el notebook de la práctica guiada. El archivo real presente en esta carpeta es

`3_memory_and_reflection.ipynb` , con otro nombre y otro alcance: implementa memoria de corto/largo plazo con reflexión sobre el *usuario*, no el patrón exacto

Reflector→Curator→playbook sobre las *propias trazas del agente* que describen las diapositivas 27-29. La relación entre ambos, y dónde exactamente diverge el código del enunciado teórico, se documenta en la §7 (Laboratorio).

Hallazgo clave de esta sesión: la diapositiva 30 ("Ya lo estás usando: Skills") describe el estándar abierto `agentskills.io` y el archivo `SKILL.md` como la versión productizada del *playbook* de ACE — una lección destilada de una sesión, serializada en Markdown, versionada en git y portable entre agentes. Ese mecanismo no es un ejemplo abstracto: es **exactamente** el sistema de *Skills* que esta propia sesión de Claude Code tiene disponible y usa (`Skill tool`, listado de *skills* del sistema). La clase describe, sin saberlo el estudiante todavía, la arquitectura del asistente con el que está aprendiendo.

1. Objetivos y agenda

Objetivos declarados — al final de la sesión, el estudiante podrá:

1. Instrumentar un agente y leer sus trazas.
2. Elegir entre LangSmith, LangFuse y *logging* propio.

3. Convertir logs en memoria útil (ciclo CRUD).
4. Implementar un agente que reescribe su propio *prompt*.

El punto de partida — una deuda de la clase anterior: la Sesión 19

(`Sesion19_FeedbackYCorreccion_ANALISIS_COMPLETO.md`) terminó mostrando agentes que se autocorrigen *dentro de una tarea* (Self-Refine, Reflexion+intérprete), pero con un problema sin resolver: la corrección se pierde cuando la tarea termina.

```
tarea 1 → reflexiona → tarea 2    (la lección se perdió)
```

La pregunta que abre esta sesión es directa: *cada ejecución deja un rastro* — *¿cómo se convierte ese rastro en una mejora que se queda?* Pero antes de aprender del rastro hay que tenerlo, y el material advierte que muchos agentes en producción simplemente no lo tienen.

El recorrido de la sesión (agenda, diapositivas 1 y 8):

#	Bloque	Pregunta que responde
1	El problema de la caja negra	¿Qué falta cuando un agente no deja rastro?
2	Ver: observabilidad	¿Cómo se instrumenta un agente y con qué herramienta?
3	Guardar: de logs a memoria	¿Cómo se destila una traza en una lección reutilizable?
4	Reescribirse: el prompt como memoria	¿Cómo se actualiza el propio <i>system prompt</i> sin perder lo aprendido?
—	Casos reales	Depuración de agentes y cierre del ciclo incidente→CI
5	Práctica	Instrumentar un agente y dejar que aprenda de su traza
—	Cierre	Síntesis y puente a la Sesión 21

Cada bloque depende del anterior: sin trazas no hay memoria, y sin memoria no hay auto-reescritura — la sesión completa es una cadena de prerequisites, no cuatro temas sueltos.

2. El problema de la caja negra (diapositivas 4-7)

Un agente sin trazas no se puede depurar, auditar ni mejorar — solo se puede reiniciar y esperar que no vuelva a fallar:

```
consulta → [ agente ] → respuesta rara
      ?
      ¿Qué herramienta falló?
      ¿Cuántos tokens gastó?
      ¿Por qué entró en loop?
```

Ninguna de esas tres preguntas es respondible sin instrumentación. El material lo plantea como una discusión de grupo (3 min), con tres preguntas que valen la pena responder honestamente para cualquier sistema real que se esté construyendo: *si diera una respuesta rara, ¿podrían saber qué paso falló?, ¿qué se está registrando ahora mismo, y quién lo mira?, ¿cuánto tardarían en detectar que el costo por petición se duplicó?*

3. Ver: observabilidad (diapositivas 9-16)

3.1 Los tres pilares

La observabilidad clásica se apoya en tres señales: **logs** (qué pasó), **métricas** (cuánto) y **trazas** (en qué orden). El material subraya que para agentes el pilar decisivo es el tercero — no basta con saber que se llamó al LLM (*Large Language Model*, modelo de lenguaje de gran escala) y a una tool; importa el **orden y el anidamiento** de esos pasos (qué llamó a qué, y desde dónde). Este modelo de datos (logs + métricas + trazas) es común a LangSmith y a Langfuse; Langfuse en particular lo expone directamente sobre OpenTelemetry (OTel), el estándar abierto de instrumentación.

3.2 Tres formas de instrumentar un agente LangChain/LangGraph

Herramienta	Cómo se conecta	Qué es
LangSmith	<i>Callback</i> nativo	Plataforma oficial de LangChain
LangFuse	Instrumentación sobre OpenTelemetry	<i>Open-core</i> , sin <i>vendor lock-in</i>
AgentTrace (o <i>logging</i> propio)	Instrumentación manual	Propuesta académica / <i>logging</i> casero

LangSmith (diapositiva 11) — *framework-agnostic* (no exige LangChain), con *tracing*, *evals*, gestión de *prompts* y *deployment*; *dashboards*, alertas y colas de anotación. Puede desplegarse en la nube, híbrido o *self-hosted* con el mismo conjunto de funciones — el material marca explícitamente que "ya no es solo cloud", contra una creencia extendida.

LangFuse (diapositivas 12-13) — modelo *open-core*: el núcleo es MIT y gratis para *self-host* sin límite de uso, salvo la carpeta `ee/` (SCIM, *audit log*, retención extendida), que requiere licencia comercial. Dato de contexto 2026 que aparece explícito en la diapositiva: **Langfuse fue adquirida por ClickHouse en enero de 2026**, manteniendo la licencia MIT del núcleo.

AgentTrace (diapositiva 14) — la "vía académica": propone subir un nivel de abstracción respecto a LangSmith/LangFuse, que en su forma más simple solo trazan la lista plana de llamadas al modelo. AgentTrace en cambio construye un **grafo de ejecución completo con relaciones padre-hijo** (un `plan` que se ramifica en `tool A`, `tool B`, un reintento, etc.), no solo una secuencia.

3.3 Recap: cuál elegir (diapositiva 15)

La tabla comparativa del material (reconstruida aquí en formato legible — el PDF original la entrega como bloques de texto sueltos sin alinear columna-fila):

Criterio	LangSmith	LangFuse	Propio
Integración LangChain	Nativa	<i>Callback</i>	Manual
Licencia	Propietaria	MIT + <code>ee/</code>	Tuya
Hosting	Cloud / híbrido / <i>self</i>	Cloud + <i>self</i>	<i>Self</i>
OpenTelemetry	Parcial	Nativo	Depende
Costo inicial	Gratis limitado	Generoso	Tu tiempo
Extensibilidad	Limitada	Total	Total

Estrategia híbrida sugerida: LangSmith en desarrollo (por el *feedback* rápido de su UI) + LangFuse en producción (por el control de datos que da el *self-host*) — no son mutuamente excluyentes.

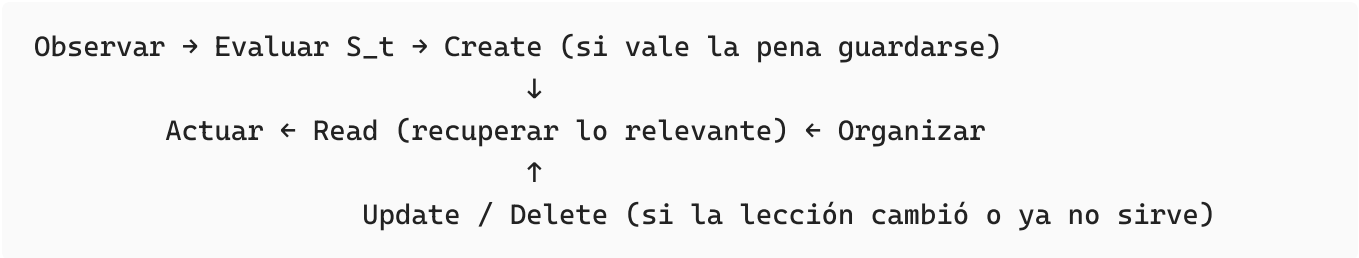
Discusión de grupo (diapositiva 16): la pregunta que decide "casi todo", según el material, es si los datos de la organización pueden salir de ella o no — y la advertencia es que si la respuesta a eso es "no lo sé", esa es la primera tarea pendiente, no elegir plataforma.

4. Guardar: de logs a memoria (diapositivas 17-21)

4.1 El ciclo de memoria guiado por señal

Con miles de trazas ya disponibles, el material propone un ciclo CRUD (*Create*, *Read*, *Update*, *Delete*) guiado por una señal — el *score* de una traza (éxito, crítica recibida, costo).

Reconstruyendo el diagrama circular de la diapositiva 18 (basado en texto disperso alrededor de un gráfico):



es lo que convierte una caché pasiva (guardar todo, sin criterio) en un motor de aprendizaje: decide si una traza vale la pena destilarse, y más adelante si esa lección sigue siendo válida. La fuente académica de este marco es Ren et al., *"Self-Improvements in Modern Agentic Systems: A Survey"* (arXiv:2607.13104, julio 2026), §6.2.3 "Memory Processing" — confirmado en la [página del paper en arXiv](#): la survey formaliza un agente moderno como un modelo base más un *scaffold* de *prompts*, memoria, *tools* y control, y el auto-mejoramiento como un operador de actualización sobre ese *scaffold*.

4.2 Los cuatro dilemas del CRUD

Operación	Si te pasas	Si te quedas corto
Create	Ruido en el <i>retrieval</i>	Pierdes capacidad a largo plazo
Read	Distraes al modelo, gastas tokens	Falla la planificación
Update	Fusiones borran detalles	Datos obsoletos persisten
Delete	Pierdes conocimiento crítico	Inundación de ruido

El error más frecuente, según el material: volcar los logs crudos en la memoria tal cual. *Create* debe ser **destilación selectiva** (extraer la lección, no archivar la traza completa), no un simple *append*. La pregunta que cierra el bloque —¿y dónde se guarda lo destilado?— es la que abre el siguiente: la respuesta moderna es guardarlo en el propio *system prompt*.

Discusión de grupo (diapositiva 21): diseñar una política de memoria propia respondiendo qué merecería guardarse como lección (C), cuándo se leería sin inyectarla siempre (R), y qué volvería obsoleta una lección y quién la borra (U/D) — con una advertencia que conecta directamente con la Sesión 19: si la lección guardada es falsa, el error se vuelve permanente, y toda regla necesita una forma de revocarse.

5. Reescribirse: el prompt como memoria (diapositivas 22-30)

5.1 De corregir la salida a corregir la instrucción

```
system prompt (pt) → ejecutar → evaluar (señal S_t) → proponer edición → pt+1
```

La distinción con la Sesión 19 es explícita: *antes* se corregía la salida (la respuesta puntual mejoraba, pero la instrucción seguía igual en la siguiente llamada); *ahora* se corrige la instrucción misma, así que la mejora se aplica a **todas** las ejecuciones futuras, no solo a la que falló. Aquí "*prompt*" se refiere específicamente al *system prompt* estable que se reutiliza en cada llamada, no a un mensaje puntual.

5.2 Cuatro paradigmas, según la riqueza de la señal

Lo que distingue a los métodos de auto-reescritura es cuánta información lleva — cuanto más rica la señal, más dirigida la edición y menos "prueba y error a ciegas":

Paradigma	Forma de la señal	Métodos
1. Score escalar	"82% de acierto"	APE, OPRO
2. Crítica textual	"falla si hay 2 productos"	Reflexion, ACE
3. Población	"de 8 prompts, ganan estos 2"	EvoPrompt, GEPA
4. Gradiente textual	"mejoraría si pidieras X"	APO, TextGrad

Con solo un número (1) hay que probar variantes y quedarse con la mejor, casi a ciegas. Con una crítica en lenguaje (2) ya se sabe qué corregir puntualmente. En (3) no se edita un solo *prompt* sino que se evolucionan varios en paralelo. En (4) la crítica viene con una dirección explícita, imitando un gradiente de optimización pero en texto. La sesión se queda deliberadamente en el paradigma 2 (crítica textual); el 3 (población, con GEPA como método estrella) queda anunciado como tema de la siguiente clase.

5.3 El enfoque ingenuo, y por qué colapsa

La pregunta inmediata al tener críticas en lenguaje natural es: ¿por qué no simplemente pedirle al LLM "aquí está el *prompt* actual más las críticas, reescríbelo mejor"? El material muestra el resultado medido de hacer exactamente eso, tomado del paper de ACE:

	Paso 60 (antes de un <i>rewrite</i> completo)	Paso 61 (después del <i>rewrite</i>)
Tokens del <i>prompt</i>	18,282	122
<i>Accuracy</i>	66.7%	57.1%

Al reescribirlo todo de golpe, el LLM **resume** en vez de **editar**: 18 mil tokens de conocimiento acumulado (casos borde, excepciones, formatos aprendidos) se evaporan en 122 tokens genéricos, y el rendimiento cae 10 puntos. El material llama a esto **colapso de contexto** (*context collapse*), y su conclusión es que la solución no es "reescribir mejor", sino dejar de reescribir el bloque completo.

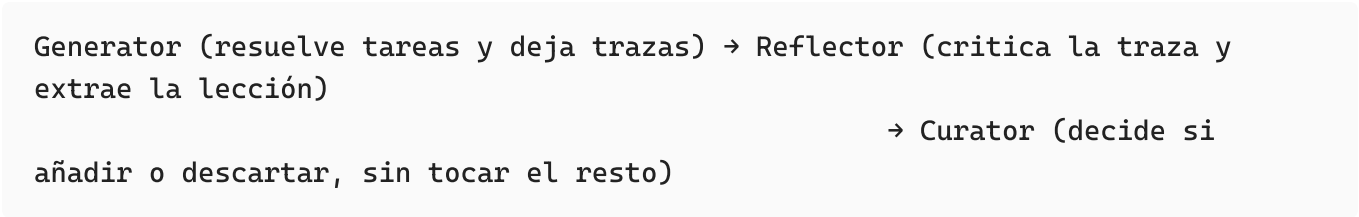
5.4 ACE: el contexto como *playbook*, no como *prompt*

ACE = Agentic Context Engineering ([Zhang et al., arXiv:2510.04618](#), octubre 2025) trata el contexto no como un texto que se reescribe entero, sino como un **manual de jugadas**: una lista de viñetas cortas e independientes, cada una con una estrategia, una trampa conocida o un formato.

Prompt monolítico	<i>Playbook</i> de viñetas
Un bloque de texto	Muchas piezas pequeñas
Actualizarlo = reescribirlo entero	Actualizarlo = añadir o editar una viñeta
Riesgo: se pierde lo aprendido	Lo demás queda intacto

Es la misma lógica que un *changelog* o un *commit* de git: cambios incrementales y trazables, no versiones nuevas desde cero. Verificado externamente: la descripción del paper coincide con lo que dice el material — ACE define explícitamente el problema del **"brevity bias"** (la tendencia a perder detalle de dominio por buscar concisión) junto al colapso de contexto, y reporta mejoras de **+10.6% en tareas de agente y +8.6% en finanzas** frente a los *baselines*, con menor latencia y costo de adaptación (fuente: [resumen en arXiv](#)).

Tres papeles, un *playbook* (diapositiva 27):



El **Curator** es la pieza clave: aplica un *delta* (añadir, editar, o nada) al *playbook* sin reescribir lo demás. Comparado contra dos *baselines* de referencia — GEPA y *Dynamic Cheatsheet* — sobre los *benchmarks* AppWorld (tareas de agente con APIs), FiNER y Formula (razonamiento financiero): **+12.5% accuracy y -82.3% latencia frente a GEPA, +7.6% accuracy y -83.6% costo frente a Dynamic Cheatsheet**.

Un ciclo completo, paso a paso (diapositivas 28-29) — ejemplo de un clasificador de tickets de soporte:

1. **La traza falla:** el ticket *"Olvidé mi password y el link de reset no llega"* se clasifica como TECNICO , cuando lo esperado era CUENTA .
2. **El Reflector extrae la regla:** *"si el problema impide acceder a la cuenta, es CUENTA aunque el síntoma sea técnico."*
3. **El Curator decide:** compara la regla contra el *playbook* actual, no está cubierta → **ADD**.

Tras varios ciclos más, el *playbook* v3 acumula tres reglas aprendidas (cuenta bloqueada, cargo no reconocido, fallo solo en un navegador), sin haber tocado ni una palabra de las dos líneas originales del *prompt*. El material también muestra el caso contrario: una candidata rechazada — el Reflector propone *"los problemas de contraseña son de CUENTA"* y el Curator responde **SKIP** porque ya la cubre la regla 1. Ese **SKIP** sistemático es también la señal de parada: si el Curator casi siempre descarta, el *playbook* ya convergió y seguir solo añade tokens a cada llamada futura sin beneficio.

5.5 Ya lo estás usando: Skills

traza de una sesión → lección destilada → SKILL.md → se carga en la siguiente sesión

Un *skill* es, en esencia, una entrada de *playbook* llevada a su forma productizada: una actualización reutilizable que se serializa en un archivo, se versiona en git y es portable entre agentes y sesiones — "el andamiaje de la clase anterior (Sesión 19), hecho texto", según el propio material.

El estándar que cita la diapositiva, agentskills.io , existe y es real: Anthropic lo publicó como estándar abierto el **18 de diciembre de 2025**, con la especificación de *SKILL.md* — un único archivo Markdown con *frontmatter* YAML (metadatos) y un cuerpo de instrucciones, opcionalmente acompañado de carpetas *scripts/* , *references/* y *assets/* . Más de 26 plataformas lo adoptaron (Claude Code, OpenAI Codex CLI, Gemini CLI, GitHub Copilot, Cursor, VS Code, entre otras) — un mismo *skill* funciona igual en cualquiera de ellas (fuentes: [Firecrawl](#), ["Agent Skills Explained"](#); [especificación en agentskills.io](https://agentskills.io)).

Por qué esto es más que una curiosidad para este documento en particular: el asistente que está redactando este análisis (Claude Code) tiene acceso, en esta misma conversación, a una lista de *skills* activables por nombre (*artifact-design* , *code-review* , *run* , etc.), cada uno respaldado por un archivo *SKILL.md* real. La diapositiva 30 no describe un concepto lejano de investigación — describe, con precisión, la pieza de infraestructura que este documento está usando para existir.

6. Casos reales: depurar y cerrar el ciclo (diapositivas 31-35)

6.1 Por qué no sirve un depurador clásico

	Programa clásico	Agente LLM
Unidad atómica	Línea de código	<i>Step function</i>
Estado	Variable en memoria	Evento de entrada/salida
Costo de repetir	Barato de re-ejecutar	Cada paso cuesta dinero
Inspeccionabilidad	Estado inspeccionable	Módulos caja negra

Depurar un agente se parece más a depurar un *pipeline* de datos que un programa tradicional: las unidades no son instrucciones baratas de repetir, son módulos opacos y caros de re-ejecutar.

6.2 LADYBUG: un depurador para agentes

Herramienta interactiva presentada en **EDBT 2025** (Barcelona, marzo 2025) para trazar, intervenir y re-ejecutar agentes LLM. Formaliza la ejecución como una traza , donde cada registra entrada, función y salida de un paso. Verificado: el paper existe, con autoría de **Rorseth, Godfrey, Golab, Srivastava y Szlichta** ([proceedings de EDBT 2025](#)), soporta agentes LlamaIndex de forma nativa, aunque el concepto aplica igual a LangGraph o CrewAI.

La diferencia clave frente a un depurador clásico es de diseño, no solo de implementación: un depurador imperativo pausa, cambia una variable en vivo y sigue; LADYBUG es **declarativo y post-hoc** — se revisa la traza completa, se encolan los cambios deseados, y solo se re-ejecutan los pasos afectados por esos cambios. Ese matiz existe por el costo: en una traza larga y cara, no es viable re-ejecutar todo tras cada prueba de depuración.

6.3 El caso del promedio imposible

Un ejemplo concreto que ilustra un segundo nivel de ayuda cuando ni el propio desarrollador encuentra el error: un agente que califica ensayos reporta un promedio de clase de **1222.92%**. El usuario revisa paso por paso y no encuentra el error. Entra un segundo LLM ("*LLM-aided debugger*"), que recibe las firmas y *docstrings* de cada paso más la traza completa, y debe: (1) decidir si la salida es incorrecta, (2) señalar el primer paso donde se rompió, (3) proponer la invocación corregida. Se localiza una alucinación en un paso tardío, se sustituye su salida (*mock*), y solo se recalcula lo que dependía de ella — no todo el flujo.

El material señala la ironía directamente: se usa un LLM para arreglar la alucinación de otro LLM.

6.4 Un incidente no debería morir en el ticket

incidente en producción → diagnóstico (traza) → caso en el dataset de eval → corre en CI para siempre

Ejemplo: un agente entra en bucle, con 20+ llamadas a la misma *tool*, con entradas ambiguas. El arreglo no es solo parchear ese caso puntual: es un límite de iteraciones, mejores descripciones de *tools*, y un detector de patrones circulares — y ese caso queda como parte del set de evaluación que corre en cada *pipeline* de integración continua. Sin ese último paso, según el material, el mismo fallo reaparece en tres meses y nadie recuerda por qué se había arreglado la primera vez.

6.5 Los límites: qué no se loguea y qué se vigila

Privacidad	Escala
Nunca loguear PII (<i>Personally Identifiable Information</i> , información personal identificable)	Muestreo en alto volumen
Anonimizar IDs	<i>Logging</i> asíncrono
Rotar <i>API keys</i>	Política de retención

El material advierte un efecto contraintuitivo: si el *logging* es síncrono, la propia observabilidad se convierte en parte de la latencia que se quería medir. Alertas mínimas sugeridas, para no vivir mirando el *dashboard*: tasa de error > 5%, latencia P99 (percentil 99) > 10 s, costo diario sobre un umbral definido.

7. Laboratorio — de la teoría al código (3_memory_and_reflection.ipynb)

Como se anota arriba (Nota técnica 2), el notebook presente en esta carpeta **no es** el `notebooks/11_logs.ipynb` que citan las diapositivas 38-39 — implementa un alcance distinto, más cercano a la §4 (memoria) que a la §5 (auto-reescritura del *prompt* estilo ACE). Vale la pena documentar exactamente dónde coincide y dónde diverge, porque las divergencias son tan instructivas como las coincidencias.

7.1 Qué implementa el notebook

Tres ejercicios progresivos, cada uno con su propia clase o función:

Ejercicio	Mecanismo	Corresponde a
Memoria corta (<code>InMemoryChatMessageHistory + RunnableWithMessageHistory</code>)	Historial de chat en RAM, por <code>session_id</code>	Memoria conversacional clásica — prerequisite, no es el foco de la sesión
Memoria larga con FAISS (<code>save_to_memory / search_memory</code>)	Cada turno se guarda como <i>embedding</i> en un índice vectorial; se recupera por similitud semántica	§4.1 (ciclo CRUD), pero solo implementa Create y Read
<code>ReflectiveMemoryAgent</code>	Cada N mensajes, un segundo LLM (<code>reflection_llm</code> , gpt-4o-mini) extrae "reflexiones" categorizadas (IDENTIDAD, PREFERENCIAS, PERSONALIDAD, RELACIONES, OBJETIVOS, HECHOS) sobre el usuario , y las guarda en un índice FAISS aparte	Un Generator/Reflector simplificado — pero sobre <i>hechos del usuario</i> , no sobre <i>errores del propio agente en una tarea</i>

7.2 Dónde el código diverge de ACE (§5.4), y por qué importa

El patrón Generator→Reflector→Curator de las diapositivas 27-29 destila lecciones sobre **cómo mejorar la ejecución de una tarea** (p. ej. una regla de clasificación de tickets) y las aplica como *delta* a un *playbook* que se **inyecta en el system prompt**.

`ReflectiveMemoryAgent` hace algo relacionado pero distinto: destila hechos sobre **quién es el usuario** (nombre, mascota, preferencias) y los recupera por búsqueda semántica para personalizar respuestas. Tiene un Reflector (la `REFLECTION_PROMPT`), pero **no tiene Curator**: no hay ningún paso que compare una reflexión nueva contra las ya guardadas y decida `ADD / SKIP / EDIT` — `_save_reflections()` simplemente añade (*Create* puro, sin deduplicar).

Esa ausencia no es un detalle menor: la salida real de la corrida incluida en el notebook (celda `f25794b6`) lo demuestra en vivo. La sección impresa "REFLEXIONES ALMACENADAS" contiene **más de 15 variantes casi idénticas** de la misma información ("El usuario se llama María y es diseñadora gráfica", "El nombre del usuario es María, es diseñadora gráfica y vive en Barcelona", "El usuario se llama María y trabaja como diseñadora gráfica"...), repetidas porque cada tanda de reflexión vuelve a extraer y guardar hechos ya guardados antes, sin comparar contra lo existente. Esto es, literalmente y sin exagerar, el **"error más frecuente"**

que advierte la diapositiva 20 ("volcar los logs crudos en la memoria... Create debe ser destilación selectiva, no *append*") y el lado izquierdo del dilema de *Create* en la tabla de la §4.2 ("ruido en el *retrieval*") — reproducido como comportamiento real y verificable en la salida del propio laboratorio, no como una advertencia abstracta de diapositiva.

De igual manera, no existe ningún método de `Update` ni `Delete` en `ReflectiveMemoryAgent`: una vez guardada, una reflexión nunca se corrige ni expira. Eso coincide exactamente con el lado derecho del dilema de *Update* en la §4.2: "datos obsoletos persisten".

7.3 Instrumentación declarada pero no conectada

El notebook importa y configura Langfuse al inicio (celda `b4b1cfbc`):

```
Langfuse(  
    public_key=os.getenv("LANGFUSE_PUBLICKEY"),  
    secret_key=os.getenv("LANGFUSE_SECRETKEY"),  
    host="https://us.cloud.langfuse.com"  
)  
langfuse = get_client()  
langfuse_handler = CallbackHandler()
```

Pero en ninguna de las llamadas posteriores (`agent_with_history.invoke(...)`, `chain.invoke(...)`, `self.agent_chain.invoke(...)` dentro de `ReflectiveMemoryAgent`) se le pasa `config={"callbacks": [langfuse_handler]}` — el *handler* queda declarado pero, a juzgar por el código tal como está escrito, nunca se conecta a una ejecución real. Es un contraste llamativo con el mensaje central de la §3 y de la síntesis final ("instrumenta desde el día 1, no cuando algo falle"): el propio material de laboratorio de esta sesión configura la instrumentación y luego no la conecta.

7.4 Lo que sí funciona como se espera

La memoria semántica con FAISS sí demuestra su punto central: en la simulación de "reinicio del programa" (crear una nueva instancia de `ReflectiveMemoryAgent`, sin historial en RAM), el agente responde correctamente preguntas como "*¿qué mascota tengo?*" o "*¿a qué me dedico?*" recuperando la información desde el índice persistido en disco (`FAISS.load_local`) — la memoria sobrevive al reinicio precisamente porque no vive solo en la memoria corta (RAM), que sí se pierde. Ese es el mismo argumento estructural de la §4.1 (Create/Read con persistencia real), aplicado correctamente, incluso si el *Update/Delete* del ciclo CRUD queda sin implementar.

8. Cierre — lo que hay que llevarse (diapositiva 42)

El hilo de la sesión, en cuatro frases (síntesis explícita del material):

1. Instrumenta desde el día 1, no cuando algo falle.
2. La traza es materia prima de aprendizaje, no solo de *debugging*.
3. Destila; no acumules logs crudos.
4. Actualiza el *prompt* con *deltas*, nunca de golpe.

Puente a la siguiente sesión: hoy un agente mejoró su *prompt* leyendo su propia historia, en solitario. La pregunta que queda abierta y que abre la Sesión 21 es: ¿y si hubiera muchos agentes, compitiendo y compartiendo lo aprendido entre sí? — la carpeta

`Sesion21_Aprendizaje_Colectivo` ya existe en este repositorio, confirmando que ese es efectivamente el tema siguiente del programa.

9. Checklist práctico — instrumentando y haciendo aprender a un agente

- ☐ ¿El agente tiene algún tipo de traza hoy? Si la respuesta es no, esa es la tarea previa a cualquier otra de esta lista.
 - ☐ ¿Los datos de las trazas pueden salir de la organización? Si no se sabe, resolver eso antes de elegir LangSmith/LangFuse/propio.
 - ☐ ¿Basta con trazar llamadas al modelo (lista plana), o hace falta el grafo de ejecución completo con relaciones padre-hijo (estilo AgentTrace)?
 - ☐ ¿Qué de lo que el sistema registra merecería guardarse como lección reutilizable (Create), y qué es solo ruido operativo?
 - ☐ ¿Cuándo se leería esa lección (Read), y cómo se evita inyectarla siempre en cada llamada, gastando tokens de más?
 - ☐ ¿Qué haría que una lección quede obsoleta (Update/Delete), y quién (o qué proceso) la borra?
 - ☐ Si el agente reescribe su propio *prompt*: ¿lo hace por *deltas* (viñetas que se añaden/editan una a una, estilo ACE) o por reescritura completa (riesgo de colapso de contexto)?
 - ☐ ¿El mecanismo de auto-mejora distingue entre "lección sobre cómo hacer la tarea mejor" y "hecho sobre el usuario/contexto"? Son memorias distintas con distinto ciclo de vida (ver §7.2).
 - ☐ ¿Existe un camino de incidente→traza→caso de evaluación→CI, o cada fallo se arregla una sola vez y se olvida?
 - ☐ ¿Se está logueando algo que no debería (PII, *API keys* en claro)? ¿El *logging* es asíncrono o compite con la latencia real del agente?
-

10. Referencias

Del material original:

- Diapositivas propias del curso (44 diapositivas), con los diagramas de observabilidad, ciclo CRUD, paradigmas de reescritura y ACE.
- Ren, Z., Chen, Y., Guo, D., et al. (2026). "Self-Improvements in Modern Agentic Systems: A Survey." [arXiv:2607.13104](https://arxiv.org/abs/2607.13104).
- Zhang, Q., et al. (2025). "Agentic Context Engineering: Evolving Contexts for Self-Improving Language Models." [arXiv:2510.04618](https://arxiv.org/abs/2510.04618).
- Agrawal, L., et al. (2026). "GEPA: Reflective Prompt Evolution Can Outperform Reinforcement Learning." ICLR 2026 (oral); preprint original [arXiv:2507.19457](https://arxiv.org/abs/2507.19457) (julio 2025).
- "AgentTrace: A Structured Logging Framework for Agent System Observability" (2026). [arXiv:2602.10133](https://arxiv.org/abs/2602.10133) — AAAI 2026 Workshop LaMAS.
- Rorseth, J., Godfrey, P., Golab, L., Srivastava, D., Szlichta, J. (2025). "LADYBUG: an LLM Agent DeBUGger for Data-Driven Applications." [EDBT 2025](https://arxiv.org/abs/2507.19457).
- Yang, C., Wang, X., Lu, Y., et al. (2023). "Large Language Models as Optimizers (OPRO)." [arXiv:2309.03409](https://arxiv.org/abs/2309.03409).
- Yuksekgonul, M., et al. (2024). "TextGrad: Automatic Differentiation via Text." [arXiv:2406.07496](https://arxiv.org/abs/2406.07496).
- [Documentación de LangSmith](#).
- [Documentación de Langfuse](#).

Investigación complementaria (verificada externamente para este documento):

- Confirmación de que ACE ([arXiv:2510.04618](https://arxiv.org/abs/2510.04618)) reporta +10.6% en agentes y +8.6% en finanzas frente a *baselines*, con el problema de "brevity bias" y colapso de contexto como motivación explícita del método — coincide con lo que enseña el material ([resumen en arXiv](#)).
- Confirmación de que GEPA fue aceptado como *oral* en ICLR 2026, con lista de autores verificada (Agrawal, Tan, Soylu, et al.) — el material cita el año de aceptación (2026), no el del *preprint* original (2025) ([arXiv:2507.19457](https://arxiv.org/abs/2507.19457)).
- Confirmación de la adquisición de Langfuse por ClickHouse, anunciada el **16 de enero de 2026** junto a una ronda Serie D de \$400M que triplicó la valoración de ClickHouse a \$15,000M — Langfuse mantiene su núcleo MIT y el *self-hosting* ([blog oficial de ClickHouse](#); [InfoWorld](#)).
- Confirmación de los precios de LangSmith citados en la diapositiva 11 (Developer gratis con 5k trazas/mes, Plus a \$39/asiento/mes) vigentes para 2026.
- Confirmación de que el estándar `agentskills.io / SKILL.md` es real, publicado por Anthropic el 18 de diciembre de 2025, adoptado por más de 26 plataformas ([Firecrawl](#),

["Agent Skills Explained"](#); [especificación oficial](#)) — y es, de hecho, el mecanismo que este mismo asistente usa en esta sesión (ver Hallazgo clave, arriba).

- Confirmación de existencia y autoría de LADYBUG (EDBT 2025) y de AgentTrace (AAAI 2026 Workshop LaMAS) como *papers* reales, no referencias inventadas.
- Arco interno del curso: `Sesion19_FeedbackYCorreccion_ANALISIS_COMPLETO.md` (Módulo 7) — esta sesión retoma explícitamente el problema de la "lección que se evapora" con el que cerraba la anterior; la carpeta `Sesion21_Aprendizaje_Colectivo` (ya presente en el repositorio) confirma el puente hacia la siguiente clase sobre aprendizaje multiagente.

Documento generado a partir del PDF de la Sesión 20 (Módulo 7, UTEC Posgrado) — 44 páginas extraídas con `pāftotext` tras encontrar el archivo protegido contra edición — más análisis del notebook de laboratorio adjunto (`3_memory_and_reflection.ipynb`) e investigación propia sobre las fuentes académicas citadas (ACE, GEPA, AgentTrace, LADYBUG, Ren et al., y el estándar Agent Skills).